

Entregable 6. Operacion, observabilidad y continuidad

6.1 Resumen

Area	Estado	Nota
SLIs y SLOs	Propuesta	Requiere metricas en produccion
Logs estructurados	Implementado	Los tres procesos, con correlacion
Metricas	Propuesta	OpenTelemetry
Trazas distribuidas	Parcial	Correlacion propia, sin OpenTelemetry
Alertas y dashboards	Propuesta	Dependen de las metricas
Health checks	Implementado	Endpoint /health, comprueba la base
Capacidad	Propuesta	Estimacion incluida
Backup y recuperacion	Propuesta	
Pruebas de fallo	Implementado	Escenarios ejecutados

6.2 SLIs y SLOs

Indicador	Como se mide	Objetivo
Disponibilidad de la API	Peticiones sin error 5xx sobre el total	99.5 % mensual
Latencia de lectura	Percentil 95 de la consulta de solicitudes	Menos de 500 ms
Latencia de registro	Percentil 95 del alta de solicitud	Menos de 1 s
Latencia del outbox	publicado_en menos creado_en	p95 menor a 5 s
Solicitudes completadas	Ejecutadas sobre terminadas	Mas del 98 %
Antigüedad de la DLQ	Mensaje mas viejo sin atender	Menos de 24 h

La latencia de registro es corta **porque el registro no espera al legado**: responde 202 y la saga continua aparte. Medir el flujo completo contra un objetivo seria medir a SICOIN, no a nosotros.

Presupuesto de error: con 99.5 % mensual son unas 3.6 horas de indisponibilidad. Si se consume mas de la mitad, se congela el despliegue de funcionalidad hasta recuperar margen.

6.3 Logs estructurados

Los tres procesos registran con ILogger y parametros nombrados, no cadenas concatenadas:

```
log.LogWarning("Reproceso {Intento} de {Maximo} sobre {Numero}, ordenado por {Usuario}.", ...);
```

Asi el motor de busqueda indexa Numero como campo y permite filtrar por solicitud, no por subcadena. En produccion la salida seria JSON hacia un agregador. Nunca se registran contraseñas, tokens, NIT ni cuentas bancarias.

6.4 Metricas

Propuestas, expuestas en formato Prometheus:

Metrica	Tipo	Para que
sigop_solicitudes_registradas_total	Contador	Volumen por entidad y tipo
sigop_transiciones_total	Contador	Cuantas terminan en cada estado
sigop_outbox_pendientes	Medidor	Si sube, el despachador no avanza
sigop_outbox_latencia_segundos	Histograma	Retraso entre escribir y publicar
sigop_mensajes_reintentados_total	Contador	Salud de las integraciones
sigop_dlq_mensajes	Medidor	Lo que requiere intervencion
sigop_legado_duracion_segundos	Histograma	Latencia de SICOIN

La metrica de pendientes del outbox es la señal de alerta temprana: crece antes de que el usuario note nada.

6.5 Trazas distribuidas

Hoy existe **correlacion propia**: la cabecera X-Correlation-Id viaja del portal a la API, se guarda en la solicitud, en cada cambio de estado y en el outbox, y se propaga en las propiedades del mensaje. La cabecera se devuelve en cada respuesta, así que un usuario puede citarla al reportar un problema.

Lo que falta para trazas completas es OpenTelemetry con traceparent del W3C, que además del identificador aporta la jerarquia de tramos y la duracion de cada salto. El punto de propagacion ya existe: se trataria de sustituir el mecanismo, no de anadirlo.

6.6 Alertas

Alerta	Condicion	Severidad
Outbox atascado	Mas de 100 pendientes durante 5 minutos	Critica
DLQ con actividad	Cualquier mensaje nuevo en la DLQ	Alta
Legado degradado	p95 de SICOIN mayor a 5 s durante 10 minutos	Alta
Tasa de error	Mas de 1 % de errores 5xx durante 5 minutos	Critica
Consumidor detenido	Sin mensajes procesados en 15 minutos con cola no vacia	Critica
Presupuesto de error	Consumido mas del 50 % del mes	Informativa

La alerta de outbox atascado cubre a la vez broker caido, despachador muerto y error de enrutamiento: los tres se manifiestan igual.

6.7 Dashboards

Operativo: solicitudes por estado, pendientes del outbox, profundidad de colas y DLQ por entidad.

Tecnico: latencias por endpoint, tasa de error y duracion de las llamadas al legado y al banco.

Negocio: volumen y monto por entidad y tipo, tasa de rechazo y tiempo medio hasta Ejecutada.

6.8 Health checks

GET /health -> 200 Healthy

Comprueba que **la base responda**, no solo que el proceso viva: un proceso arriba sin base acepta peticiones que van a fallar, y el balanceador le seguiria enviando trafico. Si PostgreSQL se detiene, el endpoint pasa a Unhealthy con 503.

En produccion se separaria en liveness, para decidir si reiniciar, y readiness, para decidir si enviar trafico, con una comprobacion adicional de RabbitMQ en los workers.

6.9 Capacidad

Recurso	Estimacion
Solicitudes	Unas 5,000 diarias, con picos a fin de mes y cierre de ejercicio
API	2 replicas; cada una absorbe el pico con holgura
Consumidor	1 por entidad de alto volumen, con prefetch 5
Outbox	2 instancias; FOR UPDATE SKIP LOCKED permite el reparto
PostgreSQL	Crecimiento dominado por historial_estados y outbox_mensajes

Las tablas outbox_mensajes y mensajes_procesados **crecen sin limite**: hay que purgar lo publicado con mas de 30 dias. No esta implementado y es la primera tarea de mantenimiento pendiente.

6.10 Backup y recuperacion

Elemento	Estrategia	RPO	RTO
PostgreSQL	Respaldo diario mas WAL continuo	5 min	1 h
RabbitMQ	Colas duraderas; los mensajes se reconstruyen desde el outbox	-	15 min
Configuracion	En control de versiones	0	Minutos
Secretos	Respaldo del gestor	0	Minutos

Perder RabbitMQ no pierde datos: los eventos estan en outbox_mensajes. Los que no llegaron a marcarse como publicados se reenvian solos, y los ya publicados y perdidos se reponen cambiando su estado a pendiente. Ese es el valor real del outbox en continuidad, y es dificil de conseguir sin el.

La restauracion se prueba trimestralmente en un entorno aparte: un respaldo que nunca se restaura no es un respaldo.

6.11 Pruebas de fallo

Ejecutadas sobre el piloto:

Escenario	Como se provoca	Resultado observado
Legado inalcanzable	Apuntar la URL a un puerto muerto	3 reintentos de 30 s, luego Fallida y DLQ, en unos 110 s
Fallo del banco	Monto igual o mayor a Q500,000	Fallida reprocesable, sin reintentos automaticos
Rechazo del legado	Monto mayor a Q1,000,000	Rechazada, estado terminal
Mensaje duplicado	Reentrega del broker	Segundo procesamiento sin efecto

Pendientes para produccion: caida de RabbitMQ con carga en curso, conmutacion de PostgreSQL y saturacion de una entidad para comprobar el aislamiento entre colas.